AI Agent 개발 실습

Google ADK로 배우는 에이전트 구축 8단계

Sense → Think → Act

단순한 챗봇에서 멀티 에이전트 오케스트레이션까지

MindCanvas Inc.

2026

목차

Lab 0 환경 설정 — Google ADK 설치 및 API 키 구성
Lab 1 첫 에이전트 만들기 — Agent 인스턴스 생성과 기본 대화
Lab 2 도구 연결하기 — Function Tool과 외부 API 연동
Lab 3 가드레일 — 안전장치와 입출력 필터링
Lab 4 **Sensing** — 환경 감지, 콜백, 관찰
Lab 5 기억 활용 — Session State와 컨텍스트 관리
Lab 6 **Loop** — Sense-Think-Act 반복과 자기 수정
Lab 7 멀티 에이전트 — Sub-Agent와 역할 분담
Lab 8 오케스트레이션 — Sequential, Parallel, Routing

관통 시나리오: AI 여행 플래너

이 실습 전체를 관통하는 하나의 프로젝트를 만들어 갑니다.

Lab 1에서 간단한 여행 추천 챗봇으로 시작하여, Lab 8에서는 항공편 검색, 숙소 예약, 일정 최적화를 병렬로 처리하는 멀티 에이전트 오케스트레이션 시스템으로 완성합니다.

💡 **핵심 개념:** 에이전트는 단순한 챗봇이 아닙니다. 환경을 감지(Sense)하고, 판단(Think)하고, 행동(Act)하는 자율적 시스템입니다. 각 Lab은 이 사이클의 한 측면을 깊이 있게 다룹니다.

Lab 0

환경 설정

Google ADK 설치 및 API 키 구성

🎯 **학습 목표**

- ✓ Python 가상환경을 만들고 Google ADK를 설치할 수 있다
- ✓ Gemini API 키를 발급받고 환경변수로 설정할 수 있다
- ✓ ADK Web UI를 실행하여 에이전트를 테스트할 수 있다

Step 1: Python 환경 준비

Python 3.10 이상이 필요합니다. 가상환경을 만들어 프로젝트를 격리합니다.

```
# 프로젝트 폴더 생성
mkdir travel-agent && cd travel-agent

# 가상환경 생성 및 활성화
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate

# Google ADK 설치
pip install google-adk
```

Step 2: API 키 설정

Google AI Studio(aistudio.google.com)에서 Gemini API 키를 발급받습니다.

```
# .env 파일 생성
echo "GOOGLE_GENAI_USE_VERTEXAI=FALSE" > .env
echo "GOOGLE_API_KEY=여기에_API_키_입력" >> .env
```

Step 3: 프로젝트 구조

아래와 같은 폴더 구조를 만듭니다. 각 Lab에서 agent.py를 수정해 나갑니다.

```
travel-agent/
├── .env
├── travel_agent/        # 에이전트 패키지
│   ├── __init__.py
```

```
│   └── agent.py         # 에이전트 정의
└── requirements.txt
```

Step 4: ADK Web UI 실행

ADK는 내장 Web UI를 제공하여 에이전트를 즉시 테스트할 수 있습니다.

```
# 프로젝트 루트에서 실행
adk web

# 브라우저에서 http://localhost:8000 접속
# 왼쪽 패널에서 에이전트를 선택하고 대화 시작
```

💡 **참고:** adk web 명령은 현재 디렉토리의 에이전트 패키지를 자동으로 감지합니다. travel_agent/ 폴더가 올바른 위치에 있는지 확인하세요.

Lab 1

첫 에이전트 만들기

Agent 인스턴스 생성과 기본 대화

🎯 **학습 목표**

- ✓ Google ADK의 Agent 클래스를 이해하고 인스턴스를 생성할 수 있다
- ✓ instruction(시스템 프롬프트)의 역할을 이해한다
- ✓ ADK Web UI 또는 코드로 에이전트와 대화할 수 있다

핵심 개념

에이전트의 가장 기본적인 구성 요소는 세 가지입니다: 이름(name), 모델(model), 그리고 지시사항(instruction)입니다. instruction은 에이전트의 역할, 성격, 행동 규칙을 정의하는 시스템 프롬프트입니다.

실습 1-1: 가장 단순한 에이전트

travel_agent/agent.py 파일을 작성합니다.

```
import google.genai
from google.adk.agents import Agent

root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="""
    당신은 친절한 여행 플래너입니다.
    사용자의 여행 계획을 도와주세요.
    한국어로 대화하며, 구체적인 추천을 제공합니다.
    """
)
```

실습 1-2: __init__.py 설정

travel_agent/__init__.py에서 에이전트를 노출합니다.

```
from . import agent
```

실습 1-3: 코드로 직접 실행하기

Web UI 외에 Python 코드에서 직접 에이전트를 호출할 수도 있습니다.

```
# run_agent.py (프로젝트 루트에 생성)
import asyncio
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types
from travel_agent.agent import root_agent

async def main():
    session_service = InMemorySessionService()
    runner = Runner(agent=root_agent, app_name="travel_app",
                    session_service=session_service)

    session = await session_service.create_session(app_name="travel_app",
                                                   user_id="user1")

    # 사용자 메시지 전송
    message = types.Content(
        role="user",
        parts=[types.Part.from_text("제주도 2박 3일 여행 추천해줘")]
    )

    response = runner.run(
        user_id=session.user_id,
        session_id=session.id,
        new_message=message,
    )

    async for event in response:
        if event.content and event.content.parts:
            print(event.content.parts[0].text)

if __name__ == "__main__":
    asyncio.run(main())
```

실습 1-4: instruction 실험

instruction을 변경하면 에이전트의 성격과 응답 방식이 완전히 달라집니다. 아래 세 가지 버전을 각각 테스트하고 차이를 비교해 보세요.

```
# 버전 A: 예산 중심 플래너
instruction = """
당신은 알뜰 여행 전문가입니다.
항상 예산을 먼저 물어보고, 가성비 좋은 옵션을 추천합니다.
```

```
숙소는 게스트하우스, 식사는 현지 맛집 위주로 제안합니다.
"""

# 버전 B: 럭셔리 컨시어지
instruction = """
당신은 프리미엄 여행 컨시어지입니다.
5성급 호텔, 미쉐린 레스토랑, 프라이빗 투어를 추천합니다.
격식 있는 존칭을 사용합니다.
"""

# 버전 C: 모험 가이드
instruction = """
당신은 모험 여행 가이드입니다.
트레킹, 다이빙, 현지 문화 체험 등 액티비티를 중심으로 추천합니다.
반말 톤으로 친근하게 대화합니다.
"""
```

💡 **핵심 포인트:** instruction은 에이전트의 '영혼'입니다. 같은 LLM이라도 instruction에 따라 완전히 다른 에이전트가 됩니다. 좋은 instruction은 역할, 제약 조건, 응답 스타일을 명확히 정의합니다.

🚀 도전 과제

1. instruction에 '응답은 반드시 3개의 추천을 포함할 것'이라는 규칙을 추가하고 테스트하세요.
2. 한국어/영어 모두 대응하는 다국어 에이전트 instruction을 작성해 보세요.
3. 에이전트에게 특정 주제(예: 정치)에 대해 답변을 거부하는 규칙을 추가해 보세요.

Lab 2

도구 연결하기

Function Tool과 외부 API 연동

🎯 **학습 목표**

- ✓ 에이전트에 도구(Tool)를 연결하여 외부 기능을 호출할 수 있다
- ✓ Python 함수를 도구로 변환하는 방법을 이해한다
- ✓ 도구의 docstring이 LLM의 도구 선택에 미치는 영향을 이해한다

핵심 개념

도구(Tool)는 에이전트가 '생각만 하는 존재'에서 '행동하는 존재'로 전환되는 핵심입니다. LLM은 도구의 이름과 설명(docstring)을 보고 어떤 도구를 언제 호출할지 스스로 판단합니다.

실습 2-1: 첫 번째 커스텀 도구

날씨 정보를 반환하는 도구를 만들어 연결합니다.

```
import google.genai
from google.adk.agents import Agent

def get_weather(city: str) -> dict:
    """특정 도시의 현재 날씨 정보를 조회합니다.

    Args:
        city: 날씨를 조회할 도시 이름 (예: '서울', '제주')

    Returns:
        온도, 상태, 습도 정보를 담은 딕셔너리
    """
    # 실습용 더미 데이터 (실제로는 API 호출)
    weather_data = {
        "서울": {"temp": 18, "condition": "맑음", "humidity": 45},
        "제주": {"temp": 22, "condition": "흐림", "humidity": 70},
        "부산": {"temp": 20, "condition": "맑음", "humidity": 55},
    }
    return weather_data.get(city, {"temp": 20, "condition": "정보없음", "humidity": 50})


root_agent = Agent(
```

```
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="""
    당신은 여행 플래너입니다.
    사용자가 여행지 날씨를 물으면 get_weather 도구를 사용하세요.
    날씨 정보를 바탕으로 적절한 옷차림과 준비물을 추천해 주세요.
    """,
    tools=[get_weather],
)
```

실습 2-2: 여러 도구 조합

에이전트에 복수의 도구를 제공하면, LLM이 상황에 맞는 도구를 자동으로 선택합니다.

```
def search_attractions(city: str, category: str) -> list:
    """도시의 관광지를 카테고리별로 검색합니다.

    Args:
        city: 검색할 도시
        category: 관광지 유형 ("자연", "문화", "맛집", "쇼핑")

    Returns:
        관광지 이름과 설명 리스트
    """
    data = {
        ("제주", "자연"): [
            {"name": "성산일출봉", "desc": "유네스코 세계자연유산"},
            {"name": "한라산", "desc": "해발 1,947m 한국 최고봉"},
        ],
        ("제주", "맛집"): [
            {"name": "흑돼지거리", "desc": "제주 특산 흑돼지 맛집 밀집"},
            {"name": "동문시장", "desc": "현지인이 사랑하는 전통시장"},
        ],
    }
    return data.get((city, category), [{"name": "정보 없음"}])


def estimate_budget(city: str, days: int, style: str) -> dict:
    """여행 예산을 추정합니다.

    Args:
        city: 여행 도시
        days: 여행 일수
        style: 여행 스타일 ("알뜰", "보통", "럭셔리")

    Returns:
```

```
        항목별 예상 비용
    """
    daily_cost = {
        "알뜰": {"숙소": 50000, "식비": 30000, "교통": 15000},
        "보통": {"숙소": 100000, "식비": 50000, "교통": 25000},
        "럭셔리": {"숙소": 250000, "식비": 100000, "교통": 50000},
    }
    costs = daily_cost.get(style, daily_cost["보통"])
    return {k: v * days for k, v in costs.items()}


root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="""
    당신은 종합 여행 플래너입니다.
    - 날씨 질문 → get_weather 사용
    - 관광지 추천 → search_attractions 사용
    - 예산 문의 → estimate_budget 사용
    도구의 결과를 종합하여 친절하게 안내합니다.
    """,
    tools=[get_weather, search_attractions, estimate_budget],
)
```

실습 2-3: Google Search 내장 도구

ADK는 Google Search를 내장 도구로 제공하여 실시간 웹 검색이 가능합니다.

```
from google.adk.tools import google_search

root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="여행 관련 최신 정보가 필요하면 검색하세요.",
    tools=[google_search, get_weather, search_attractions],
)
```

💡 **핵심 포인트:** 도구의 docstring은 LLM이 읽는 '사용 설명서'입니다. 설명이 모호하면 LLM이 잘못된 도구를 선택하거나, 필요한 도구를 사용하지 않습니다. Args의 타입과 설명을 정확히 작성하세요.

🚀 **도전 과제**

1. docstring을 일부러 모호하게 바꿔보고, 에이전트가 도구를 잘못 사용하는 경우를 관찰하세요.
2. 환율 계산 도구(convert_currency)를 추가하여 해외여행 예산 추정을 해 보세요.

3. 도구가 에러를 반환하는 경우를 시뮬레이션하고, 에이전트가 어떻게 대응하는지 관찰하세요.

Lab 3

가드레일

안전장치와 입출력 필터링

🎯 **학습 목표**

- ✓ before_model_callback을 사용하여 입력을 필터링할 수 있다
- ✓ after_model_callback을 사용하여 출력을 검증할 수 있다
- ✓ 도구 호출 전후에 가드레일을 적용할 수 있다

핵심 개념

도구를 연결한 에이전트는 실제 행동(API 호출, 데이터 조회 등)을 수행합니다. 이제 '어떤 행동을 허용하고 어떤 행동을 막을 것인가'라는 안전 설계가 필요합니다. ADK는 콜백 시스템으로 이를 구현합니다.

실습 3-1: 입력 필터링 (before_model_callback)

모델에 메시지가 전달되기 전에 부적절한 내용을 차단합니다.

```
from google.adk.agents import Agent
from google.genai import types

BLOCKED_KEYWORDS = ['도박', '마약', '불법']

def input_guardrail(callback_context, llm_request):
    """모델 호출 전 입력을 검사합니다."""
    user_message = llm_request.contents[-1].parts[0].text

    for keyword in BLOCKED_KEYWORDS:
        if keyword in user_message:
            # 차단된 경우 직접 응답을 반환
            return types.Content(
                role="model",
                parts=[types.Part.from_text(
                    f"죄송합니다. 해당 주제에 대해서는 도움을 드릴 수 없습니다."
                )]
            )

    return None  # None 반환 = 통과, 정상 진행
```

```
root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="당신은 여행 플래너입니다.",
    before_model_callback=input_guardrail,
)
```

실습 3-2: 출력 검증 (after_model_callback)

모델의 응답에서 민감한 정보가 포함되었는지 확인합니다.

```
import re

def output_guardrail(callback_context, llm_response):
    """모델 응답 후 출력을 검사합니다."""
    if not llm_response or not llm_response.content:
        return None

    text = llm_response.content.parts[0].text

    # 개인정보(전화번호 패턴) 마스킹
    masked = re.sub(r"\d{3}-\d{4}-\d{4}", "[전화번호 마스킹]", text)

    if masked != text:
        llm_response.content.parts[0].text = masked

    return llm_response


root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="당신은 여행 플래너입니다.",
    before_model_callback=input_guardrail,
    after_model_callback=output_guardrail,
)
```

실습 3-3: 도구 호출 가드레일

특정 도구의 호출을 조건부로 허용하거나 차단합니다.

```
def tool_guardrail(callback_context, tool_name, tool_args):
    """도구 호출 전 검사합니다."""
    if tool_name == "estimate_budget":
```

```
        days = tool_args.get("days", 0)
        if days > 30:
            return {
                "error": "30일을 초과하는 여행은 지원하지 않습니다."
            }
    return None  # 정상 진행


root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="당신은 여행 플래너입니다.",
    tools=[get_weather, estimate_budget],
    before_tool_callback=tool_guardrail,
)
```

💡 **핵심 포인트:** 가드레일은 도구를 연결한 직후에 설계해야 합니다. 도구는 에이전트에게 '행동할 능력'을 주고, 가드레일은 '행동의 범위'를 정합니다. 이 두 가지는 항상 쌍으로 설계하세요.

🚀 도전 과제

1. 특정 국가(예: 분쟁 지역)에 대한 여행 추천을 차단하는 가드레일을 만들어 보세요.
2. 1일 예산이 100만원을 초과하면 경고 메시지를 추가하는 출력 가드레일을 만드세요.
3. before/after 콜백의 실행 로그를 파일로 저장하는 감사(audit) 시스템을 만들어 보세요.

Lab 4

Sensing

환경 감지, 콜백, 관찰

🎯 **학습 목표**

- ✓ 에이전트의 Sense-Think-Act 사이클을 이해한다
- ✓ 도구 실행 결과를 관찰하고 판단하는 패턴을 구현할 수 있다
- ✓ 외부 환경 변화를 감지하여 에이전트 행동에 반영할 수 있다

핵심 개념

Sensing은 에이전트가 세상을 '인식'하는 방법입니다. 단순한 사용자 입력 외에도, 도구 실행 결과, API 응답, 시간 변화, 데이터 상태 등 다양한 환경 신호를 감지할 수 있습니다.

Sense (감지)	Think (판단)	Act (행동)
도구 결과 관찰 환경 데이터 수집 사용자 의도 파악	상황 분석 다음 행동 결정 우선순위 판단	도구 호출 응답 생성 상태 업데이트

실습 4-1: 도구 결과 관찰 패턴

after_tool_callback을 사용하여 도구의 실행 결과를 관찰하고, 필요시 에이전트의 행동을 조정합니다.

```
import json

def observe_tool_result(callback_context, tool_name, tool_args, tool_result):
    """도구 실행 후 결과를 관찰하고 상태를 업데이트합니다."""

    # 날씨 결과 관찰 → 악천후 감지
    if tool_name == "get_weather":
        result = json.loads(str(tool_result)) if isinstance(tool_result, str) else
tool_result
        condition = result.get("condition", "")

        if condition in ["태풍", "폭우", "폭설"]:
            # 세션 상태에 경고 플래그 설정
            callback_context.state["weather_alert"] = True
```

```
            callback_context.state["alert_type"] = condition
            print(f"⚠️ 악천후 감지: {condition}")

    # 예산 결과 관찰 → 고비용 감지
    if tool_name == "estimate_budget":
        result = json.loads(str(tool_result)) if isinstance(tool_result, str) else
tool_result
        total = sum(result.values()) if isinstance(result, dict) else 0
        if total > 2000000:
            callback_context.state["high_budget_alert"] = True
            print(f"💰 고비용 감지: {total:,}원")

    return None  # 결과는 변경하지 않고 관찰만


root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="""
    당신은 여행 플래너입니다.
    weather_alert 상태가 True이면 실내 활동을 우선 추천하세요.
    high_budget_alert가 True이면 절약 팁을 함께 제공하세요.
    """,
    tools=[get_weather, search_attractions, estimate_budget],
    after_tool_callback=observe_tool_result,
)
```

실습 4-2: 환경 컨텍스트 센서

현재 시간, 날짜, 시즌 등 환경 정보를 감지하여 에이전트의 추천에 반영합니다.

```
from datetime import datetime

def get_travel_context() -> dict:
    """현재 여행 컨텍스트를 감지합니다.

    Returns:
        현재 시간, 계절, 여행 성수기 여부 등 환경 정보
    """
    now = datetime.now()
    month = now.month

    # 계절 판단
    seasons = {
        (3,4,5): "봄", (6,7,8): "여름",
        (9,10,11): "가을", (12,1,2): "겨울"
    }
```

```
    season = next(v for k, v in seasons.items() if month in k)

    # 성수기 판단
    peak = month in [7, 8, 12, 1]

    return {
        "current_date": now.strftime("%Y-%m-%d"),
        "season": season,
        "is_peak_season": peak,
        "recommendation": "성수기입니다. 조기 예약을 권장합니다." if peak else "비수기라 할인
혜택이 많습니다."
    }


root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="""
    당신은 여행 플래너입니다.
    여행 추천 전에 반드시 get_travel_context를 호출하여
    현재 계절과 성수기 여부를 확인하고, 이를 반영하여 추천하세요.
    """,
    tools=[get_travel_context, get_weather, search_attractions],
)
```

실습 4-3: 다중 신호 종합 판단

여러 센서의 결과를 종합하여 판단하는 패턴입니다. 날씨 + 시즌 + 예산을 모두 고려한 추천을 생성합니다.

```
def analyze_conditions(city: str, days: int, budget_style: str) -> dict:
    """여러 조건을 종합 분석하여 여행 적합성을 판단합니다.

    Args:
        city: 여행 도시
        days: 여행 일수
        budget_style: 예산 스타일

    Returns:
        종합 분석 결과와 권장 사항
    """
    weather = get_weather(city)
    context = get_travel_context()
    budget = estimate_budget(city, days, budget_style)

    # 종합 점수 산출
```

```
    score = 100
    warnings = []

    if weather["condition"] in ["태풍", "폭우"]:
        score -= 40
        warnings.append("악천후 예보")
    if context["is_peak_season"]:
        score -= 15
        warnings.append("성수기 가격 상승")
    if sum(budget.values()) > 1500000:
        score -= 10
        warnings.append("높은 예산")

    return {
        "suitability_score": score,
        "warnings": warnings,
        "recommendation": "추천" if score >= 60 else "재검토 권장",
        "details": {"weather": weather, "season": context, "budget": budget}
    }
```

💡 **핵심 포인트:** Sensing은 Loop의 전제 조건입니다. 에이전트가 '무엇이 변했는지'를 감지할 수 있어야, '다시 시도할지'를 판단할 수 있습니다. Lab 6에서 이 sensing 결과를 기반으로 반복 루프를 구현합니다.

🚀 도전 과제

1. 특정 관광지의 혼잡도를 시뮬레이션하는 센서를 만들어 보세요 (시간대별로 다른 값 반환).
2. 환율 변동을 감지하여 해외여행 예산을 자동 조정하는 센서를 만들어 보세요.
3. 여러 도시의 조건을 동시에 감지하고, 가장 적합한 도시를 추천하는 에이전트를 만드세요.

Lab 5

기억 활용

Session State와 컨텍스트 관리

🎯 **학습 목표**

- ✓ 세션(Session)과 상태(State)의 개념을 이해한다
- ✓ 대화 간 정보를 기억하고 활용할 수 있다
- ✓ 사용자 선호도를 저장하여 개인화된 응답을 생성할 수 있다

핵심 개념

기본적으로 LLM은 이전 대화를 기억하지 못합니다. ADK의 Session과 State 시스템을 활용하면, 에이전트가 대화 맥락과 사용자 정보를 유지할 수 있습니다.

실습 5-1: 세션 상태로 선호도 저장

사용자가 밝힌 선호도를 세션 상태에 저장하고 이후 대화에서 활용합니다.

```
def save_preference(category: str, value: str,
                    tool_context) -> dict:
    """사용자의 여행 선호도를 저장합니다.

    Args:
        category: 선호도 유형 ("음식", "숙소", "활동", "예산")
        value: 선호도 값 (예: "한식", "호텔", "액티비티", "보통")

    Returns:
        저장 결과
    """
    # 세션 상태에 선호도 저장
    prefs = tool_context.state.get("user_preferences", {})
    prefs[category] = value
    tool_context.state["user_preferences"] = prefs

    return {"status": "저장완료", "category": category, "value": value}


def get_preferences(tool_context) -> dict:
    """저장된 사용자 선호도를 조회합니다.
```

```
    Returns:
        저장된 선호도 딕셔너리
    """
    return tool_context.state.get("user_preferences", {})


root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="""
    당신은 개인 맞춤형 여행 플래너입니다.

    대화 규칙:
    1. 새로운 사용자가 오면 선호도를 자연스럽게 파악하세요.
    2. 파악된 선호도는 save_preference로 저장하세요.
    3. 추천 전에 get_preferences로 저장된 선호도를 확인하세요.
    4. 선호도에 맞춰 개인화된 추천을 제공하세요.
    """,
    tools=[save_preference, get_preferences,
           get_weather, search_attractions],
)
```

실습 5-2: 대화 이력 활용

이전 대화에서 언급된 도시나 일정을 기억하고 참조합니다.

```
def log_destination(city: str, dates: str,
                    tool_context) -> dict:
    """여행 목적지를 기록합니다.

    Args:
        city: 방문 예정 도시
        dates: 여행 날짜 (예: "2026-05-01~03")
    """
    history = tool_context.state.get("travel_history", [])
    history.append({"city": city, "dates": dates})
    tool_context.state["travel_history"] = history
    return {"logged": True, "total_destinations": len(history)}


def get_itinerary_summary(tool_context) -> dict:
    """현재까지 계획된 여행 일정을 요약합니다."""
    history = tool_context.state.get("travel_history", [])
    prefs = tool_context.state.get("user_preferences", {})
    return {
        "destinations": history,
        "preferences": prefs,
```

```
        "total_cities": len(history)
    }
```

실습 5-3: 상태 기반 대화 흐름 제어

대화 단계를 상태로 관리하여 체계적인 여행 계획 수립을 유도합니다.

```
root_agent = Agent(
    name="travel_assistant",
    model="gemini-2.0-flash",
    instruction="""
    당신은 체계적인 여행 플래너입니다.

    대화 단계를 state['phase']로 관리하세요:
    - "greeting": 인사 및 여행 목적 파악
    - "preference": 선호도 수집 (음식, 숙소, 활동, 예산)
    - "planning": 구체적 일정 계획
    - "review": 최종 검토 및 확정

    각 단계의 필수 정보가 모이면 다음 단계로 전환하세요.
    현재 단계를 사용자에게 알려주세요.
    """,
    tools=[save_preference, get_preferences,
           log_destination, get_itinerary_summary,
           get_weather, search_attractions, estimate_budget],
)
```

💡 **핵심 포인트:** State는 에이전트의 '작업 기억(Working Memory)'입니다. 도구 실행 결과, 사용자 선호도, 대화 단계 등을 저장하여 에이전트가 맥락 있는 대화를 이어갈 수 있게 합니다.

🚀 도전 과제

1. 사용자가 '처음부터 다시'라고 하면 모든 상태를 초기화하는 기능을 만들어 보세요.
2. 여행 이력이 3개 이상이면 패턴을 분석하여 새로운 여행지를 추천하는 기능을 만드세요.
3. 대화 시작 시 이전 세션의 선호도를 자동으로 불러오는 기능을 구현해 보세요.

Lab 6

Loop

Sense-Think-Act 반복과 자기 수정

🎯 **학습 목표**

- ✓ 에이전트가 결과를 평가하고 스스로 재시도하는 패턴을 구현할 수 있다
- ✓ LoopAgent를 활용한 반복 워크플로우를 설계할 수 있다
- ✓ 탈출 조건(exit condition)을 설정하여 무한 루프를 방지할 수 있다

핵심 개념

Loop는 Sensing에서 배운 '감지'를 반복적으로 수행하는 것입니다. 에이전트가 자신의 출력을 평가하고, 만족스럽지 않으면 다시 시도하는 자기 개선 사이클입니다. ADK의 LoopAgent는 이 패턴을 구조화합니다.

실습 6-1: 자기 평가 패턴

에이전트가 생성한 여행 일정의 품질을 스스로 평가하고 개선합니다.

```
def evaluate_itinerary(itinerary: str) -> dict:
    """생성된 여행 일정의 품질을 평가합니다.

    Args:
        itinerary: 평가할 여행 일정 텍스트

    Returns:
        점수와 개선 포인트
    """
    score = 0
    feedback = []

    # 필수 요소 체크
    checks = {
        "숙소": "숙소 정보가 포함되어야 합니다",
        "식사": "식사 계획이 포함되어야 합니다",
        "교통": "이동 수단 정보가 포함되어야 합니다",
        "시간": "시간대별 일정이 포함되어야 합니다",
        "비용": "예상 비용이 포함되어야 합니다",
    }
```

```
    for keyword, msg in checks.items():
        if keyword in itinerary:
            score += 20
        else:
            feedback.append(msg)

    return {
        "score": score,
        "passed": score >= 80,
        "feedback": feedback,
        "message": "합격!" if score >= 80 else "개선이 필요합니다"
    }


root_agent = Agent(
    name="travel_planner",
    model="gemini-2.0-flash",
    instruction="""
    당신은 완벽주의 여행 플래너입니다.

    작업 프로세스:
    1. 여행 일정을 생성합니다.
    2. evaluate_itinerary로 품질을 평가합니다.
    3. 점수가 80점 미만이면 feedback을 반영하여 일정을 수정합니다.
    4. 80점 이상이 될 때까지 반복합니다.
    5. 최대 3회까지만 반복합니다.
    """,
    tools=[evaluate_itinerary, get_weather, search_attractions],
)
```

실습 6-2: LoopAgent 활용

ADK의 LoopAgent를 사용하면 구조화된 반복을 구현할 수 있습니다.

```
from google.adk.agents import Agent, LoopAgent

# 일정 생성 에이전트
planner = Agent(
    name="planner",
    model="gemini-2.0-flash",
    instruction="""
    여행 일정을 생성하세요.
    이전 피드백이 있다면 반영하여 개선하세요.
    state에 저장된 feedback을 확인하세요.
    """,
    tools=[get_weather, search_attractions],
)
```

```
# 품질 검증 에이전트
reviewer = Agent(
    name="reviewer",
    model="gemini-2.0-flash",
    instruction="""
    생성된 일정을 평가하세요.
    evaluate_itinerary 도구로 점수를 확인하세요.
    점수가 80 이상이면 state["loop_complete"]를 True로 설정하세요.
    80 미만이면 개선 피드백을 state에 저장하세요.
    """,
    tools=[evaluate_itinerary],
)

# 루프 에이전트: planner → reviewer 반복
loop_agent = LoopAgent(
    name="planning_loop",
    sub_agents=[planner, reviewer],
    max_iterations=3,
)
```

실습 6-3: 점진적 개선 패턴

매 반복마다 구체적인 개선 이력을 추적합니다.

```
def track_improvement(iteration: int, score: int,
                      changes: str, tool_context) -> dict:
    """개선 이력을 추적합니다.

    Args:
        iteration: 현재 반복 횟수
        score: 현재 점수
        changes: 이번 반복에서 변경한 내용
    """
    history = tool_context.state.get("improvement_history", [])
    history.append({
        "iteration": iteration,
        "score": score,
        "changes": changes
    })
    tool_context.state["improvement_history"] = history

    # 진행 상황 출력
    if len(history) >= 2:
        prev_score = history[-2]["score"]
        delta = score - prev_score
        return {"improved_by": delta, "total_iterations": len(history)}
```

```
    return {"first_attempt": True, "score": score}
```

💡 **핵심 포인트:** Loop의 핵심은 '탈출 조건'입니다. 무한 루프를 방지하려면 반드시 max_iterations을 설정하고, 품질 기준(threshold)을 명확히 정의하세요. 3회 반복이 일반적인 적정선입니다.

🚀 도전 과제

1. 사용자가 '더 자세히'라고 요청할 때마다 일정의 세밀도를 높이는 점진적 루프를 만드세요.
2. 반복 횟수가 늘어날수록 평가 기준을 높이는 '적응형 루프'를 설계해 보세요.
3. 두 에이전트가 서로의 일정을 교차 평가하는 '피어 리뷰' 루프를 만들어 보세요.

Lab 7

멀티 에이전트

Sub-Agent와 역할 분담

🎯 **학습 목표**

- ✓ Sub-Agent를 정의하고 root agent와 연결할 수 있다
- ✓ 에이전트 간 역할 분담과 위임(delegation) 패턴을 이해한다
- ✓ LLM 기반 동적 라우팅으로 적절한 sub-agent를 선택할 수 있다

핵심 개념

멀티 에이전트는 하나의 복잡한 에이전트 대신, 전문화된 여러 에이전트가 협력하는 구조입니다. 마치 여행사에서 항공팀, 호텔팀, 투어팀이 각자의 전문성으로 협업하는 것과 같습니다.

실습 7-1: 전문가 에이전트 팀 구성

```
from google.adk.agents import Agent

# 항공편 전문 에이전트
flight_agent = Agent(
    name="flight_expert",
    model="gemini-2.0-flash",
    instruction="""
    당신은 항공편 전문가입니다.
    - 출발지, 도착지, 날짜를 확인합니다
    - 직항/경유, 항공사별 특징을 안내합니다
    - 예산에 맞는 항공편을 추천합니다
    항공편과 관련 없는 질문은 다른 전문가에게 위임하세요.
    """,
    tools=[search_flights],
)

# 숙소 전문 에이전트
hotel_agent = Agent(
    name="hotel_expert",
    model="gemini-2.0-flash",
    instruction="""
    당신은 숙소 전문가입니다.
    - 호텔, 리조트, 게스트하우스 등을 추천합니다
    - 위치, 편의시설, 가격대를 비교합니다
```

```
    - 후기 기반 추천을 제공합니다
    """,
    tools=[search_hotels],
)

# 현지 투어 전문 에이전트
activity_agent = Agent(
    name="activity_expert",
    model="gemini-2.0-flash",
    instruction="""
    당신은 현지 활동/투어 전문가입니다.
    - 관광지, 맛집, 액티비티를 추천합니다
    - 시간대별 효율적 동선을 제안합니다
    - 현지 팁과 주의사항을 알려줍니다
    """,
    tools=[search_attractions, get_weather],
)

# 총괄 에이전트 (라우터 역할)
root_agent = Agent(
    name="travel_coordinator",
    model="gemini-2.0-flash",
    instruction="""
    당신은 여행 총괄 코디네이터입니다.
    사용자의 요청을 파악하여 적절한 전문가에게 위임하세요:
    - 항공편 관련 → flight_expert
    - 숙소 관련 → hotel_expert
    - 관광/활동 관련 → activity_expert
    종합 일정 요청 시 모든 전문가의 결과를 조합하세요.
    """,
    sub_agents=[flight_agent, hotel_agent, activity_agent],
)
```

실습 7-2: 에이전트 간 정보 공유

State를 통해 sub-agent 간에 정보를 공유합니다.

```
# 항공편 에이전트가 도착 시간을 state에 저장
def save_flight_info(arrival_time: str, airport: str,
                     tool_context) -> dict:
    """선택된 항공편 정보를 저장합니다."""
    tool_context.state["flight_info"] = {
        "arrival_time": arrival_time,
        "airport": airport
    }
    return {"saved": True}
```

```
# 숙소 에이전트가 항공편 정보를 참조하여 추천
hotel_agent = Agent(
    name="hotel_expert",
    model="gemini-2.0-flash",
    instruction="""
    숙소를 추천할 때 state['flight_info']의 도착 공항을
    확인하고, 공항에서 가까운 숙소를 우선 추천하세요.
    """,
    tools=[search_hotels],
)
```

실습 7-3: 동적 라우팅

Root Agent는 LLM이 사용자 의도를 분석하여 자동으로 적절한 sub-agent를 선택합니다. 이것이 단순한 if-else 라우팅이 아닌 LLM 기반 지능형 라우팅의 핵심입니다.

```
# 사용자: "제주도 갈 건데 맛있는 거 먹고 싶어"
# → LLM이 분석: 관광/활동 관련 → activity_expert로 라우팅

# 사용자: "김포에서 제주 가는 비행기 알아봐줘"
# → LLM이 분석: 항공편 관련 → flight_expert로 라우팅

# 사용자: "제주 2박 3일 전체 일정 짜줘"
# → LLM이 분석: 종합 요청 → 순차적으로 전체 sub-agent 호출
```

💡 **핵심 포인트:** 멀티 에이전트의 핵심은 '관심사의 분리'입니다. 각 에이전트는 자신의 전문 영역만 담당하고, root agent가 조율합니다. 이렇게 하면 각 에이전트의 instruction이 짧고 명확해져서 성능이 향상됩니다.

🚀 **도전 과제**

1. 예산 관리 전문 에이전트(budget_expert)를 추가하고 전체 비용을 조율하게 하세요.
2. sub-agent가 서로에게 추가 정보를 요청하는 패턴을 구현해 보세요.
3. 동일 요청을 단일 에이전트 vs 멀티 에이전트로 처리하고 품질을 비교해 보세요.

Lab 8

오케스트레이션

Sequential, Parallel, Routing

🎯 **학습 목표**

- ✓ SequentialAgent로 순차 워크플로우를 구현할 수 있다
- ✓ ParallelAgent로 병렬 처리를 구현할 수 있다
- ✓ 복합 오케스트레이션 패턴을 설계할 수 있다

핵심 개념

오케스트레이션은 멀티 에이전트의 실행 순서와 방식을 제어합니다. 오케스트라의 지휘자가 각 악기의 연주 시점을 결정하듯, 에이전트들의 협업 방식을 설계합니다.

패턴	설명	사용 시점
Sequential	A → B → C 순차 실행	이전 결과가 다음 입력에 필요할 때
Parallel	A, B, C 동시 실행	독립적 작업을 빠르게 처리할 때
Loop	반복 실행	품질 기준 충족까지 개선할 때
LLM 라우팅	LLM이 동적으로 선택	사용자 의도에 따라 분기할 때

실습 8-1: Sequential (순차 처리)

항공편 → 숙소 → 활동 순서로 일정을 조립합니다. 각 단계의 결과가 다음 단계의 입력이 됩니다.

```
from google.adk.agents import Agent, SequentialAgent

# Step 1: 요구사항 분석
analyzer = Agent(
    name="requirement_analyzer",
    model="gemini-2.0-flash",
    instruction="""
    사용자의 여행 요구사항을 분석하여 구조화하세요.
    결과를 state에 저장: destination, dates, budget, interests
    """
)

# Step 2: 일정 생성
```

```
planner = Agent(
    name="itinerary_planner",
    model="gemini-2.0-flash",
    instruction="""
    state의 요구사항을 바탕으로 상세 일정을 생성하세요.
    시간대별 일정, 숙소, 식사, 교통을 포함합니다.
    """,
    tools=[get_weather, search_attractions, estimate_budget],
)

# Step 3: 최종 리뷰
reviewer = Agent(
    name="final_reviewer",
    model="gemini-2.0-flash",
    instruction="""
    생성된 일정을 최종 검토합니다.
    빠진 항목, 비현실적 일정, 예산 초과를 체크하고
    사용자에게 깔끔하게 정리된 최종 일정을 제시하세요.
    """
)

# 순차 실행: analyzer → planner → reviewer
root_agent = SequentialAgent(
    name="travel_pipeline",
    sub_agents=[analyzer, planner, reviewer],
)
```

실습 8-2: Parallel (병렬 처리)

독립적인 작업을 동시에 실행하여 처리 속도를 높입니다.

```
from google.adk.agents import Agent, ParallelAgent, SequentialAgent

# 병렬로 실행할 전문 에이전트들
flight_search = Agent(
    name="flight_search",
    model="gemini-2.0-flash",
    instruction="state의 정보로 항공편을 검색하세요.",
    tools=[search_flights],
)

hotel_search = Agent(
    name="hotel_search",
    model="gemini-2.0-flash",
    instruction="state의 정보로 숙소를 검색하세요.",
    tools=[search_hotels],
)
```

```
activity_search = Agent(
    name="activity_search",
    model="gemini-2.0-flash",
    instruction="state의 정보로 관광지를 검색하세요.",
    tools=[search_attractions],
)

# 세 에이전트를 병렬 실행
parallel_search = ParallelAgent(
    name="parallel_search",
    sub_agents=[flight_search, hotel_search, activity_search],
)
```

실습 8-3: 복합 오케스트레이션

Sequential + Parallel + Loop를 조합한 완전한 여행 플래닝 파이프라인입니다. 이것이 이 실습의 최종 완성형입니다.

```
from google.adk.agents import (Agent, SequentialAgent,
                               ParallelAgent, LoopAgent)

# Phase 1: 요구사항 분석 (단일)
analyzer = Agent(
    name="analyzer",
    model="gemini-2.0-flash",
    instruction="사용자 요구사항을 분석하여 state에 저장",
)

# Phase 2: 병렬 검색 (병렬)
parallel_search = ParallelAgent(
    name="search_all",
    sub_agents=[flight_search, hotel_search, activity_search],
)

# Phase 3: 일정 조합 + 검증 루프 (루프)
composer = Agent(
    name="composer",
    model="gemini-2.0-flash",
    instruction="검색 결과를 종합하여 일정을 조합하세요.",
)

quality_check = Agent(
    name="quality_checker",
    model="gemini-2.0-flash",
    instruction="""
    일정 품질을 평가하세요.
```

```
    80점 이상이면 state["loop_complete"] = True
    """,
    tools=[evaluate_itinerary],
)

refinement_loop = LoopAgent(
    name="refine_loop",
    sub_agents=[composer, quality_check],
    max_iterations=3,
)

# Phase 4: 최종 프레젠테이션 (단일)
presenter = Agent(
    name="presenter",
    model="gemini-2.0-flash",
    instruction="""
    최종 일정을 보기 좋게 포맷하여 제시하세요.
    일별 일정표, 총 예산, 준비물 체크리스트를 포함합니다.
    """
)

# 전체 파이프라인: 분석 → 병렬검색 → 반복개선 → 발표
root_agent = SequentialAgent(
    name="travel_orchestrator",
    sub_agents=[analyzer, parallel_search,
                refinement_loop, presenter],
)
```

💡 **최종 아키텍처:** 이 파이프라인은 Lab 1~8의 모든 개념을 통합합니다: Agent(Lab 1) + Tools(Lab 2) + Guardrails(Lab 3) + Sensing(Lab 4) + Memory(Lab 5) + Loop(Lab 6) + Multi-Agent(Lab 7) + Orchestration(Lab 8)

🚀 도전 과제

1. ParallelAgent의 결과를 비교 표로 만드는 summarizer 에이전트를 추가하세요.
2. 사용자 피드백을 받아 특정 Phase만 다시 실행하는 '부분 재실행' 기능을 구현하세요.
3. 전체 파이프라인에 가드레일과 sensing을 추가하여 프로덕션급 시스템으로 완성하세요.

부록: 학습 로드맵

Lab	핵심 질문	배우는 것	연결 개념
1	에이전트란?	Agent, Instruction	LLM 기초
2	어떻게 행동하나?	Tool, Function	Function Calling
3	어디까지 허용?	Callback, Filter	AI Safety
4	무엇을 감지?	Sensor, Observer	Perception
5	무엇을 기억?	State, Session	Context Window
6	어떻게 개선?	Loop, Evaluation	Self-Reflection
7	어떻게 협력?	Sub-Agent, Delegation	Division of Labor
8	어떻게 조율?	Orchestration	System Design

각 Lab은 약 60~90분 분량으로 설계되었습니다. 전체 과정은 2~3일(12~18시간) 집중 워크숍 또는 8주 주간 수업으로 운영할 수 있습니다.